# Data Models

## Property Graphs

Tarapong Sreenuch

8 February 2026

克明峻德，格物致知

# Building On: Topics 2, 8, and 9

- **Topic 2:** Associative entities — OrderLine carries `quantity` and `unit_price` because the data describes the Order–Product *relationship*, not either entity
- **Topic 8–9:** RDF triples, SPARQL pattern matching, property paths — multi-hop traversal exists but cannot filter at each hop, and cycle detection requires manual workarounds

Topic 9 closed with a candid gap: SPARQL property paths traverse multiple hops but cannot inspect data on each hop. This chapter fills that gap — with a question regulators actually ask.

# A Compliance Team's Two Questions

A fintech compliance team monitors account transfers. The data lives in a relational database — accounts, transfers, timestamps, amounts — normalised and constrained in the style of Topics 2–5.

Two questions arrive from regulators:

1. "Starting from flagged account A-2049, trace all outgoing transfers within **3 hops** where each transfer exceeds **$5,000** and occurred in the **last 90 days**."
2. "Are there any **cycles** — money that left an account and returned to it through a chain of intermediaries?"

Both require traversing relationships and inspecting data on each relationship at every step.

# Failure 1: SQL Recursive CTE

```
WITH RECURSIVE trace AS (
  SELECT t.to_id, t.amount, t.txn_date, 1 AS depth,
         CAST(t.from_id AS CHAR(500)) AS path
  FROM Transfer t
  WHERE t.from_id = 'acct_2049'
    AND t.amount > 5000
    AND t.txn_date > DATE_SUB(NOW(), INTERVAL 90 DAY)
  UNION ALL
  SELECT t.to_id, t.amount, t.txn_date, tr.depth + 1,
         CONCAT(tr.path, ',', t.from_id)
  FROM Transfer t JOIN trace tr ON t.from_id = tr.to_id
  WHERE tr.depth < 3 AND t.amount > 5000
    AND t.txn_date > DATE_SUB(NOW(), INTERVAL 90 DAY)
    AND FIND_IN_SET(t.to_id, tr.path) = 0
)
SELECT * FROM trace;
```

Three problems compound: edge filter **duplicated** in base and recursive case, depth tracking **manual** (`tr.depth + 1`), cycle prevention is a **string-manipulation hack** (CAST + CONCAT + FIND_IN_SET) that breaks on edge cases. The compliance officer cannot trace the logic.

# Failure 2 — RDF Reification Explosion

The transfer relationship needs properties — amount, timestamp, type. In RDF, that requires **reification**: each transfer becomes an intermediate node with 6 triples:

```
ex:txn_001 a ex:Transfer ;
    ex:fromAccount ex:acct_2049 ;
    ex:toAccount   ex:acct_3017 ;
    ex:amount      10000 ;
    ex:txnDate     "2024-03-15" ;
    ex:type        "wire" .
```

50,000 transfers → **300,000+ triples**. A 3-hop query requires 3 intermediate nodes × 6 triple patterns = **18 patterns** in WHERE. SPARQL property paths (`ex:transferredTo+`) cannot filter by amount at each hop — they follow all transfers or none. On the payment processor supernode (12,000 edges), the query **times out after 4 minutes**.

# Shared Root Cause

| Failure | Model | Relationship treatment | Why traversal breaks |
|---|---|---|---|
| SQL recursive CTE | Relational | FK (pointer) or junction table (separate entity) | Self-joins per hop; manual depth + string-hack cycle prevention |
| RDF reification | Knowledge graph | Predicate (label only — no edge data without reification) | 6 triples per edge; property paths can't filter per hop |

Both models can **store** the data. Neither makes traversal with per-edge filtering a **first-class operation**.

*How do we model relationships as data-carrying entities and traverse them with per-hop filtering — declaratively, without manual BFS loops or reification machinery?*

# Therefore: Four Capabilities Required

> Therefore, the engineer must **model relationships as first-class data carriers and traverse them with per-hop filtering** — not build BFS loops in SQL strings or route through reification intermediaries.

| # | Capability | Failure prevented |
|---|---|---|
| 1 | **Relationships as data-carrying entities** | Failure 2: reification tax (6 triples per edge) |
| 2 | **Bounded multi-hop traversal with per-hop filtering** | Failure 1: CTE opacity; Failure 2: no per-hop filtering |
| 3 | **Native cycle detection** | Failure 1: string-hack cycle prevention |
| 4 | **Supernode handling without explosion** | Failure 2: property path explosion on supernode |

# Property Graph Vocabulary

| Term | Definition |
|---|---|
| **Property graph** | Directed graph $G = (V, E, \lambda, \sigma)$ where both nodes and edges carry label + property map |
| **Node** | Entity with a label and key-value properties |
| **Edge / Relationship** | Directed connection with a label, source, target, and key-value properties |
| **Property map** | Key-value pairs on a node or edge (e.g., `{amount: 8000, date: '2024-03-01'}`) |
| **Variable-length path** | Pattern matching 1..k hops: `(a)-[*1..k]→(b)` |
| **Bounded traversal** | Traversal with explicit stopping conditions (depth, edge predicate, node predicate) |
| **Supernode** | Node with disproportionately high degree (e.g., payment processor with 12,000 edges) |

# The Financial Transfer Dataset

All queries run against a fraud detection network: a small hand-traceable **core subgraph** (8 accounts, 9 edges) for worked examples, and a surrounding network (60 accounts, 150 edges) for the lab.

```
Graph schema:

(:Account {id, name, type, flagged})
    -[:TRANSFERRED_TO {amount, date, transfer_type}]→
(:Account)
```

One node label, one relationship type. All relationship data — amount, date, transfer type — lives **on the edge**, not on either endpoint. This is the property graph equivalent of Topic 2's associative entity (OrderLine).

# Core Subgraph: The Traversal Map

```
acct_2049 (Alice, FLAGGED)
 ├──[$8K wire √]──→ acct_3017 (Bob)
 │                   ├──[$12K wire √]──→ acct_7801 (David)
 │                   │                    └──[$7K wire √]──→ acct_2049 (Ring
1)
 │                   └──[$2K ACH X]───→ acct_9900 (Elena)
 │
 └──[$3K ACH X]───→ acct_4522 (Carla)
                     └──[$6K card √]──→ processor_001 (QuickPay, SUPERNODE)
                                         ├──[$15K wire √]──→ acct_7801
                                         ├──[$9K wire √]───→ acct_5566 (Frank)
                                         └──[$1K ACH X]───→ acct_8800 (Grace)

√ = passes $5K filter    X = fails $5K filter
```

8 accounts, 9 edges. Edge 9 closes **Ring 1** by returning to acct_2049 at depth 3. processor_001 is the **supernode** — 42 edges in the full dataset (23 outgoing, 19 incoming).

# The Formal Model: G = (V, E, λ, σ)

```
Node (v ∈ V):                                  Edge (e ∈ E):

┌──────────────────────────────┐      ┌──────────────────────────────────────┐
│  (:Account)                  │      │  -[:TRANSFERRED_TO]→                 │
│  {id: 'acct_2049',           │      │  {amount: 8000,                      │
│   name: 'Alice Chen',        │      │   date: date('2024-03-01'),          │
│   type: 'Personal',          │      │   transfer_type: 'wire'}             │
│   flagged: true}             │      │  source: acct_2049  target: acct_3017│
└──────────────────────────────┘      └──────────────────────────────────────┘

  λ(v) = :Account                       λ(e) = :TRANSFERRED_TO
  σ(v) = {id, name, type, flagged}      σ(e) = {amount, date, transfer_type}
```

The critical distinction: **edges are first-class entities with their own properties**. The property map on the edge ({amount, date, transfer_type}) is what makes property graphs distinct — this data would require a junction table in the relational model or a reification node in RDF.

# Relationship Treatment Across Four Models

| Concept | Relational (Topic 2) | Document (Topic 6) | RDF (Topic 8) | Property graph (Topic 10) |
|---|---|---|---|---|
| "Alice transferred $10K to Bob" | Junction table row | Embedded subdoc or reference | Reification node with 4+ triples | Edge with properties |
| Where does relationship data live? | Separate table | Nested object or separate collection | Intermediate node | **On the edge itself** |
| Storage units per transfer | 1 row × 5 columns | 1 subdocument | **6 triples** | 1 edge × 3 properties |
| Traversal cost for k hops | k−1 self-joins or recursive CTE | Not applicable | k intermediate nodes × 6 patterns | k edge-hops, filtered in-place |

**Predict:** Given a transfer "Bank A sent $50,000 to Bank B on January 10 via wire" — where does each piece of data live: on node A, node B, or the edge? Why can it not live on either node?

# The Answer: Relationship Data Lives on the Edge

```
(A:Account)-[:TRANSFERRED_TO {amount: 50000, date: date('2024-01-10'),
transfer_type: 'wire'}]→(B:Account)
```

The amount, date, and type live on the edge because they describe the **transfer** — not Bank A, not Bank B. Bank A sends many transfers with different amounts; Bank B receives many with different amounts. The data is specific to the relationship instance.

Same argument as Topic 2: `quantity` and `unit_price` live on OrderLine because they describe the Order–Product relationship, not either entity. Three models, one principle — **relationship data belongs on the relationship**.

# Cypher Syntax: The Core Building Blocks

Cypher uses ASCII-art syntax that visually mirrors graph structure:

```
Node pattern:          (n:Label {prop: value})
Relationship pattern: -[r:REL_TYPE {prop: value}]→
Path pattern:          (a)-[r]→(b)
Variable-length:       (a)-[*1..3]→(b)        — 1 to 3 hops
Named path:            p = (a)-[*1..3]→(b)    — binds entire path to p
```

For traversal, the critical constructs:

| Construct | Purpose | Cypher |
|---|---|---|
| Depth bound | Limit how far | `*1..3` |
| Edge predicate | Limit which edges | `ALL(r IN relationships(p) WHERE ...)` |
| Node predicate | Skip intermediate nodes | `NONE(n IN nodes(p)[1..-1] WHERE ...)` |
| Cycle detection | Start = end | `(a)-[*3..6]→(a)` |

# Bounded Traversal: The "Follow the Money" Query

```
MATCH p = (start:Account {id: 'acct_2049'})-[:TRANSFERRED_TO*1..3]→(dest)
WHERE ALL(r IN relationships(p)
          WHERE r.amount > 5000 AND r.date > date('2024-01-02'))
RETURN dest.id, length(p) AS depth,
       [n IN nodes(p) | n.id] AS path
```

```
(start:Account {id:'acct_2049'})
        │
        │  [:TRANSFERRED_TO *1..3]          ← depth bound
        │  ALL(r WHERE r.amount > 5000      ← per-hop filter
        │           AND r.date > '2024-01-02')
        ▼
(dest)  ──→  RETURN dest.id, length(p)
```

Five meaningful lines. The query reads like the English requirement. Compare to the 20-line CTE and the 25-line SPARQL query.

# Node Predicates: Skipping the Supernode

```
MATCH p = (start:Account {id: 'acct_2049'})-[:TRANSFERRED_TO*1..3]→(dest)
WHERE ALL(r IN relationships(p) WHERE r.amount > 5000)
  AND NONE(n IN nodes(p)[1..-1] WHERE n.type = 'Processor')
RETURN dest.id, length(p) AS depth
```

`NONE(n IN nodes(p)[1..-1] WHERE ... )` checks **intermediate** nodes only (not start or end). `[1..-1]` is list slicing: all elements except first and last.

**Predict:** If processor_001 is at depth 2 on a path from acct_2049 and has 23 outgoing edges, what happens when the NONE clause rejects it? How many edges are saved?

# Node Predicates: The Supernode Shortcut

> All 23 outgoing edges from processor_001 are **never examined**. The entire subtree is cut.

Node predicates complement edge predicates — edge filters reduce *which edges* are followed; node filters eliminate *which expansions* happen.

# Bounded BFS: What Happens Under the Hood

When Cypher executes `*1..3`, the engine runs a bounded traversal internally:

```
BOUNDED-BFS(graph, start, max_depth, edge_predicate):
    queue = [(start, 0, path=[], used_edges={})]
    results = []

    while queue not empty:
        current, depth, path, used = dequeue(queue)
        if depth == max_depth: continue            ← depth bound
        for each edge (current → neighbour):
            if edge in used: continue              ← relationship uniqueness
            if not edge_predicate(edge): continue ← per-hop filter
            add (neighbour, depth+1) to results and queue

    return results
```

Key design choice: **no visited-node set** — nodes can be revisited. Instead, **relationship uniqueness**: no edge traversed twice in a single path. This prevents infinite loops while allowing cycles to be found.

# Tracing the Query: Predict

Recall the core subgraph (Slide 10) — edges that pass the $5K filter marked ✓:

```
acct_2049 ──[$8K ✓]──→ acct_3017 ──[$12K ✓]──→ acct_7801 ──[$7K ✓]──→ acct_2049
                        └──[$2K ✗]──→ acct_9900
acct_2049 ──[$3K ✗]──→ acct_4522 ──[$6K ✓]──→ processor_001
                                              ├──[$15K ✓]──→ acct_7801
                                              ├──[$9K ✓]──→ acct_5566
                                              └──[$1K ✗]──→ acct_8800
```

**Predict:** How many result rows will the "follow the money" query (Slide 15) return on this subgraph? Which accounts appear as destinations? Will the start account appear in results?

# Depth 1: Edges from acct_2049

```
Depth 1 — edges from acct_2049:

Edge | To            | Amount | >$5K? | Result
─────┼───────────────┼────────┼───────┼──────────────────────
```

Edge #1 passes → **(acct_3017, depth 1)** added to results.

Edge #2 fails ($3K < $5K) → the **entire Carla → processor_001 subtree is pruned**. processor_001's 3 outgoing edges are never touched. One failed edge at depth 1 prevented 4 edges from being checked.

# Depths 2 and 3: Following the Surviving Path

```
Depth 2 — from acct_3017:                    Depth 3 — from acct_7801:

Edge | To          | Amount   | Result       Edge | To          | Amount | Result
─────┼─────────────┼──────────┼──────────      ─────┼─────────────┼────────┼──────────
```

| dest.id | depth | path | Amounts along path |
|---|---|---|---|
| acct_3017 | 1 | [acct_2049, acct_3017] | [$8K] |
| acct_7801 | 2 | [acct_2049, acct_3017, acct_7801] | [$8K, $12K] |
| acct_2049 | 3 | [acct_2049, acct_3017, acct_7801, acct_2049] | [$8K, $12K, $7K] |

**3 result rows.** Only 5 of 9 edges checked — the per-hop filter reduced work by 44%. Row 3 is **Ring 1** — the money-flow cycle.

# Bounded vs Unbounded: Why Both Bounds Matter

| Metric | With Edge Filter | Without Edge Filter | Ratio |
|---|---|---|---|
| Edges checked | 5 of 9 | 9 of 9 | 1.8× |
| Result rows | 3 | 9 | 3× |
| Nodes reached | 3 unique | 8 unique | 2.7× |
| processor_001 expanded? | **No** (pruned at depth 1) | Yes (3 outgoing) | — |

Depth bounds and edge predicates serve **different** purposes:

- **Depth bounds** protect against **unbounded exploration** — without `*1..k`, the engine traverses the entire reachable graph
- **Edge predicates** protect against **irrelevant path proliferation** — without `ALL( ... WHERE ... )`, every edge is followed including low-value transfers through supernodes

Both are needed: depth limits *how far*, edge predicates limit *which edges*.

# What About Cycles?

The trace found acct_2049 at depth 3 — the money returned to its origin. But that was a side effect of following paths forward. What if the compliance team asks: "Find all money-flow rings of length 3 to 6"?

> **Predict:** In SQL, cycle prevention required CAST + CONCAT + FIND_IN_SET — 3 lines of string manipulation. In SPARQL, $O(k^2)$ inequality checks. How many lines of Cypher does cycle detection require?

# Cycle Detection: One-Line Pattern

```
MATCH p = (a:Account)-[:TRANSFERRED_TO*3..3]→(a)
WHERE ALL(r IN relationships(p) WHERE r.amount > 3000)
RETURN [n IN nodes(p) | n.id] AS ring
```

The pattern `(a)-[*3..3]→(a)` — same variable on both ends — matches cycles. No separate algorithm needed.

On the core subgraph, Ring 1 produces **3 raw rows** — one per starting node:

| Start | Path | Amounts |
|---|---|---|
| acct_2049 | acct_2049 → acct_3017 → acct_7801 → acct_2049 | $8K, $12K, $7K |
| acct_3017 | acct_3017 → acct_7801 → acct_2049 → acct_3017 | $12K, $7K, $8K |
| acct_7801 | acct_7801 → acct_2049 → acct_3017 → acct_7801 | $7K, $8K, $12K |

Three rotations of the **same ring**. Deduplicate in the lab. Bottleneck (min edge) = **$7,000**.

# How Cycles Are Treated Across Models

| Model | Cycle Is Typically… | Detection Approach |
|---|---|---|
| Relational (Topic 2–5) | Design error (circular FK) | Avoided by schema design; self-join for intentional self-reference |
| RDF (Topic 8–9) | Identity artefact (`owl:sameAs` loops) | Resolved during data cleaning — not a query target |
| Property graph (Topic 10) | **Query objective** (fraud ring) | One-line Cypher pattern: `(a)-[*k..k]→(a)` |

In fraud detection, a cycle is the **signal** — the whole point of the analysis is to find money that returns to its origin.

# Supernode Explosion: The d^k Discipline

A node with degree d at depth k produces up to **d^k** candidate paths:

| Degree (d) | k = 1 | k = 2 | k = 3 | Context |
|---|---|---|---|---|
| 5 | 5 | 25 | 125 | Typical personal account |
| 100 | 100 | 10,000 | 1,000,000 | Active business account |
| 12,000 | 12,000 | 144,000,000 | **$1.7 \times 10^{12}$** | Payment processor |

**Predict:** processor_001 has 23 outgoing edges in the full dataset. If 25% pass an amount filter, what is the effective degree? Estimate the path count at depth 2. Would you run this without further filtering?

# Layered Filtering Reduces d^k

> Effective degree = 23 × 0.25 ≈ 6. Depth-2 cost: $6^2 = 36$ — manageable. Without the filter: $23^2 = 529$. At depth 3: $6^3 = 216$ vs $23^3 = 12{,}167$ — a **56× difference** from one filter.

Concrete cost estimation for processor_001:

| Strategy | Eff. degree | Depth 2 | Depth 3 |
|---|---|---|---|
| No filter | 23 | 529 | 12,167 |
| Amount > $5K (13 of 23 pass) | 13 | 169 | 2,197 |
| Amount > $5K + date filter | 8 | 64 | **512** |
| Amount > $5K + skip Processor | 13 | 169 | — (skipped) |

**Discipline:** always estimate d^k before running a traversal. If the estimate exceeds ~100,000 paths, tighten bounds. This is Topic 4's predict-before-execute principle applied to graph traversal.

# Three Languages: Same Question

The "follow the money" query in all three languages makes the trade-offs concrete:

| Aspect | SQL (recursive CTE) | SPARQL (Reified) | Cypher |
|---|---|---|---|
| Lines of code | ~20 | ~25+ | **~6** |
| Depth control | Manual counter (`mt.depth + 1`) | Hardcoded hop blocks (copy-paste) | `*1..3` |
| Changing 3 hops → 5 | Add recursive logic | Copy-paste 2 blocks + $O(k^2)$ inequalities | Change one integer |
| Per-hop edge filter | Duplicated in base + recursive case | FILTER per hop block | One `ALL( … WHERE … )` |
| Cycle prevention | String hack (CAST / CONCAT / FIND_IN_SET) | Manual FILTER with $O(k^2)$ inequalities | Built-in relationship uniqueness |
| Readability | Opaque | Verbose | Reads like English |

# SQL vs Cypher: Side by Side

```
-- SQL: ~20 lines, duplicated filters, string-hack cycle prevention
WITH RECURSIVE money_trail AS (
  SELECT t.to_id, 1 AS depth, CAST(t.from_id AS CHAR(500)) AS path
  FROM Transfer t WHERE t.from_id = 'acct_2049' AND t.amount > 5000 ...
  UNION ALL
  SELECT t.to_id, mt.depth + 1, CONCAT(mt.path, ',', t.from_id)
  FROM Transfer t JOIN money_trail mt ON t.from_id = mt.to_id
  WHERE mt.depth < 3 AND t.amount > 5000 ...
    AND FIND_IN_SET(t.to_id, mt.path) = 0)  -- string hack
SELECT * FROM money_trail;
```

```
-- Cypher: 5 lines, one filter clause, built-in cycle safety
MATCH p = (start:Account {id: 'acct_2049'})-[:TRANSFERRED_TO*1..3]→(dest)
WHERE ALL(r IN relationships(p)
          WHERE r.amount > 5000 AND r.date > date('2024-01-02'))
RETURN dest.id, length(p) AS depth, [n IN nodes(p) | n.id] AS path
```

Cypher's advantages are concentrated in **traversal-specific operations**. SQL's advantages remain in constraint enforcement and multi-table aggregation.

# The Six-Step Traversal Procedure

| Step | Action | Parallel |
|---|---|---|
| 1 | **Identify the traversal question** — start, edge type, direction, goal | Topic 9: identify the pattern |
| 2 | **Choose bounds** — depth `*1 .. k`, edge predicates, node predicates, time window | Topic 9: write the BGP |
| 3 | **Estimate cost** — check degree, compute d^k with effective degree. If > 100K, tighten | *New* — like Topic 5 index selectivity |
| 4 | **Execute on small subgraph** — verify result matches prediction | Topic 4: predict row count |
| 5 | **Scale up and validate** — compare result-set size against estimate | Topic 4: run and verify |
| 6 | **Compare in SQL and SPARQL** — what does each make easy vs awkward? | Topic 9: cross-model comparison |

Steps 1–2 mirror Topic 9's predict-first method. Step 3 (d^k cost estimation) is the new addition.

# Procedure Applied: processor_001 Reachable Set

**Question:** "From processor_001, find all accounts reachable within 2 hops where each transfer exceeds $3,000."

**Step 1:** Start = processor_001. Edge = `:TRANSFERRED_TO`. Direction = outgoing. Goal = reachable set.

**Step 2:** Depth `*1..2`. Edge predicate: `r.amount > 3000`.

```
MATCH p = (start:Account {id: 'processor_001'})-[:TRANSFERRED_TO*1..2]→(dest)
WHERE ALL(r IN relationships(p) WHERE r.amount > 3000)
RETURN dest.id, length(p) AS depth
```

**Step 3:** 23 outgoing edges; 17 of 23 exceed $3K → effective degree ≈ 17. Cost: $17^2$ = 289 candidate paths — well within 100K threshold. Safe to execute.

**Step 4 (subgraph):** 3 outgoing edges (#6: $15K ✓, #7: $9K ✓, #8: $1K ✗). Depth 1: acct_7801, acct_5566. Depth 2: acct_7801 → acct_2049 ($7K ✓). **Predicted: 3 rows.**

# Five Data Models: Updated Comparison

| Model | Primary workload | Relationship treatment | Traversal support | Topic |
|---|---|---|---|---|
| Relational | Constraint enforcement + aggregation | FK or junction table | Recursive CTE (manual, brittle) | 2–5 |
| Document | Hierarchical reads + schema flexibility | Embedded or referenced | Not applicable | 6–7 |
| RDF | Identity + cross-source integration | Predicate (label only) | Property paths (no per-hop filtering) | 8–9 |
| **Property graph** | **Traversal + path analytics** | **First-class edge with properties** | **Cypher: declarative bounded traversal** | **10** |

**Cost accepted:** the property graph model excels at traversal but does not natively enforce the normalised, multi-table constraint patterns from Topics 2–5.

# Enforcement Spectrum: Where Does the Property Graph Sit?

The property graph model sits **outside** the main enforcement spectrum from Topic 5:

| Position | Model | What it enforces |
|---|---|---|
| Engine rejects | Relational (Topic 2–5) | NOT NULL, CHECK, FK, ACID transactions |
| Validator rejects | XML/XSD (Topic 7) | Schema validation at trust boundary |
| Nothing rejects | RDF (Topic 8) | Social contract only |
| Nothing rejects | Document (Topic 6) | Application-managed (optional JSON Schema) |
| *Outside spectrum* | **Property graph (Topic 10)** | Optional: property existence + type constraints (Neo4j Enterprise) |

The model's strength is **traversal**, not constraint enforcement. Topic 11 places all five models on this spectrum.

# Check 1: Edge Properties Are Per-Instance

A tax authority records three penalty assessments from Inspector Garcia to Corporation X:

```
(Garcia)-[:ASSESSED {amount: 50000, date: '2024-01-15', type: 'late_filing'}]→(CorpX)
(Garcia)-[:ASSESSED {amount: 20000, date: '2024-03-01', type: 'underreport'}]→(CorpX)
(Garcia)-[:ASSESSED {amount: 80000, date: '2024-03-20', type: 'fraud'}]→(CorpX)
```

Can `amount` live on the Garcia node? Can it live on the (Garcia, CorpX) pair?

> No — Garcia.amount would need to hold three values simultaneously. The node pair also has three distinct amounts. Each edge is an **independent relationship instance** with its own properties. Property graphs permit **multi-edges** (multiple edges between the same node pair). A filter `amount > 30000` correctly returns 2 of 3 assessments.

# Check 2: Supernode Cost Estimation

A hospital referral network has a central specialist (Dr Patel) with 4,000 incoming referral edges and 200 outgoing. An analyst runs:

```
MATCH p = (start:Doctor {id: 'dr_patel'})-[:REFERRED*1..3]→(dest)
RETURN dest.id, length(p) AS depth
```

Estimate the worst-case path count at depth 3 without filtering.

> 200 outgoing edges → d^k = $200^3$ = **8,000,000** candidate paths. At depth 2: $200^2$ = 40,000 — borderline. Add an edge filter (`referral_date > '2024-01-01'`) matching 30% of edges: effective degree 60. Cost: $60^3$ = **216,000** — still above 100K. Add speciality filter (10% of remainder): effective degree 6. Cost: $6^3$ = **216** — manageable. **Layered filtering is cumulative.**

# Check 3: Path Pruning (The Whole-Path Rule)

A ride-sharing network:

```
Hub ──[$50 ✓]──→ A ──[$40 ✓]──→ B
 │
 └──[$8 ✗]──→ C ──[$60 ✓]──→ D ──[$45 ✓]──→ B
```

Query: `MATCH p = (hub)-[:TRIP*1..3]→(dest) WHERE ALL(r IN relationships(p) WHERE r.fare > 20) RETURN dest`

D → B ($45) and C → D ($60) both pass the filter. Will B appear twice in the results?

> **No.** The path Hub → C → D → B fails at its first edge ($8 < $20). `ALL(r IN relationships(p) WHERE ... )` checks **every edge in the path**, not individual edges in isolation. Because the $8 edge fails, the entire C → D → B subtree is pruned — even though every subsequent edge passes. The filter prunes **paths**, not edges.

# Check 4: Relationship Uniqueness vs Node Uniqueness

A pharma supply chain has a cycle: Manufacturer → Distributor → Hospital → Manufacturer (returned defective batch).

```
MATCH p = (m:Org {id: 'pharma_mfg'})-[:SHIPPED*3..3]→(m)
RETURN [n IN nodes(p) | n.id] AS ring
```

Will Cypher find this cycle? Would a textbook BFS with a visited-node set find it?

> **Cypher: yes.** Edges used = {e1, e2, e3} — all distinct → relationship-unique. Node visited: Manufacturer appears twice → not node-unique. Cypher enforces **relationship uniqueness** (no edge reused), not node uniqueness. A textbook BFS with a visited-node set would **suppress** the cycle — Manufacturer already visited, final step blocked. When cycle detection is the goal, relationship uniqueness is the correct default.

# Check 5: Cycle Threshold Sensitivity

An e-commerce platform detects a return-fraud ring: Customer A → Merchant B → Reseller C → Customer A. Amounts: $900, $850, $750. The fraud team runs:

```
MATCH p = (a)-[:TRANSFERRED*3..3]→(a)
WHERE ALL(r IN relationships(p) WHERE r.amount > 800)
RETURN [n IN nodes(p) | n.id] AS ring
```

> The $750 edge (C → A) **fails** the $800 threshold. The ring is **not detected**. The bottleneck (minimum edge) = $750 determines the maximum threshold below which the ring becomes visible. Lowering to `amount > 700` would find it. **The bottleneck edge governs detectability** — same principle as Ring 2 in the dataset, which is detectable at `> $3K` but not at `> $5K`.

# Check 6: Cross-Model Trade-Off

A music streaming service needs two capabilities:

1. **Royalty calculation:** For each artist, compute total streams × per-stream rate, grouped by album, with contractual minimums enforced by CHECK constraints and cascade deletes when a contract expires.

2. **Influence tracing:** Starting from a trending artist, trace "listeners also liked" chains 4 hops deep, filtering by genre similarity > 0.7 at each hop, to build a recommendation path.

Which model handles which workload?

> **(1) Relational model** — normalised schema with FK constraints, CHECK constraints for contractual minimums, GROUP BY for per-album aggregation, CASCADE for contract expiry. This is Topics 2–5 territory. **(2) Property graph model** — artists as nodes, "listeners also liked" as edges with `genre_similarity` property, `*1..4` bounded traversal with `ALL(r WHERE r.genre_similarity > 0.7)`. This is Topic 10 territory. **Cost accepted:** the property graph cannot enforce contractual minimums; the relational model resists 4-hop traversal with per-hop filtering.

# Earlier We Asked: Traversal with Per-Edge Filtering?

In Slide 6, the two failures shared a root cause: relationships treated as second-class citizens. The four capabilities demanded a model where edges carry data and traversal filters on that data at every step.

The property graph model — $G = (V, E, \lambda, \sigma)$ with first-class edges — met all four requirements:

| Requirement | How the property graph delivers |
|---|---|
| Relationships as data carriers | Edge property map: `{amount, date, transfer_type}` |
| Bounded multi-hop traversal | `*1..k` with `ALL( ... WHERE ... )` |
| Native cycle detection | `(a)-[*k]→(a)` — one-line pattern |
| Supernode handling | d^k estimation + NONE clause for node predicates |

The compliance officer who couldn't trace the CTE can read the Cypher query and follow it.

# Concepts Planted Here That Recur

| Concept | Introduced Here As | Recurs In |
|---|---|---|
| Property graph edge with properties | Relationship as first-class data carrier | Topic 11 (traversal workload assignment) |
| Bounded traversal (`*1..k`) | Depth-limited path pattern | Topic 11 (cross-model query comparison) |
| Per-hop edge filtering | `ALL(r IN relationships(p) WHERE ...)` | Topic 11 (query expressiveness comparison) |
| Cycle detection as pattern | `(a)-[*k]→(a)` — same variable, both ends | Topic 11 (fraud detection use case) |
| d^k cost estimation | Predict-before-execute for graph traversal | Topic 11 (performance reasoning) |
| Relationship uniqueness | Cypher's default — edges not reused, nodes revisitable | Topic 11 (traversal semantics comparison) |
| Three-language comparison | SQL CTE vs SPARQL reified vs Cypher | Topic 11 (capstone comparison table) |

# Course Map

```
Part 1 — Relational Foundation
├── Ch 1  Introduction to Data Models
├── Ch 2  Conceptual Modelling (ERD)
├── Ch 3  Normalisation (BCNF & Star Schemas)
├── Ch 4  SQL Querying (FWGHOS Pipeline)
└── Ch 5  Enforcement (Constraints, Access Control, Transactions, Indexes)

Part 2 — Alternative Models
├── Ch 6  Document Model (JSON & MongoDB)
├── Ch 7  XML for Contracts (XSD & XPath)
├── Ch 8  RDF and Knowledge Graphs
├── Ch 9  SPARQL Querying
└── Ch 10 Property Graphs (Cypher) ◄── you are here

Part 3 — Capstone
└── Ch 11 Multi-Model Design
```

Topic 10 completes the set of four data models. Topic 11 hands you a system with multiple workloads and asks: which model handles which part? The comparison tables from this chapter and the lab's Q13–Q14 are direct input to that capstone exercise.

# Lab Preview

**Lab 10 — Property Graphs: Cypher Traversal on a Financial Transfer Network**

A code-based lab using Cypher queries against the 60-account, 150-transfer fraud network in Neo4j on Google Colab.

- **Parts 1–2** (~30 min): Load and inspect — supernode identification, 1-hop neighbourhood exploration
- **Parts 3–4** (~35 min): Multi-hop traversal with per-hop filtering, cycle detection and deduplication
- **Part 5** (~15 min): Supernode handling — edge filtering vs node predicate strategies
- **Parts 6–7** (~30 min): Cross-model comparison (SQL CTE, SPARQL reified, Cypher) and four-model synthesis

**Format:** Code-based lab in Google Colab. Predict-first discipline throughout. **Total ~110 min.**

File: `10-lab-property-graphs-cypher-traversal.ipynb`